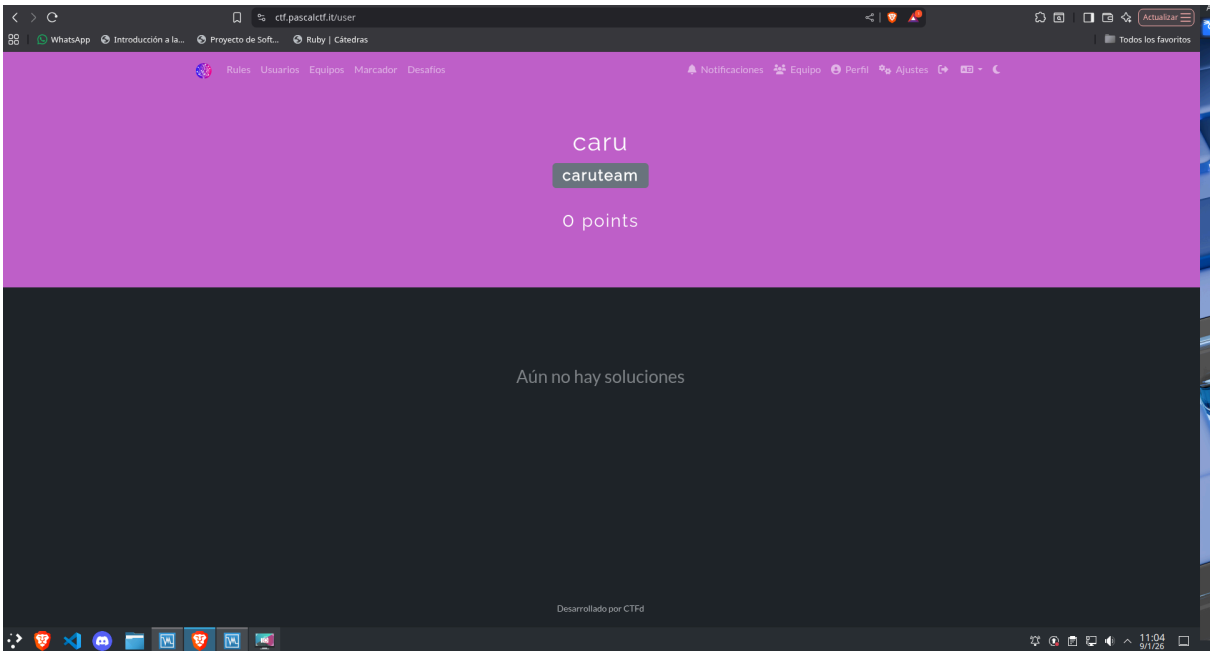
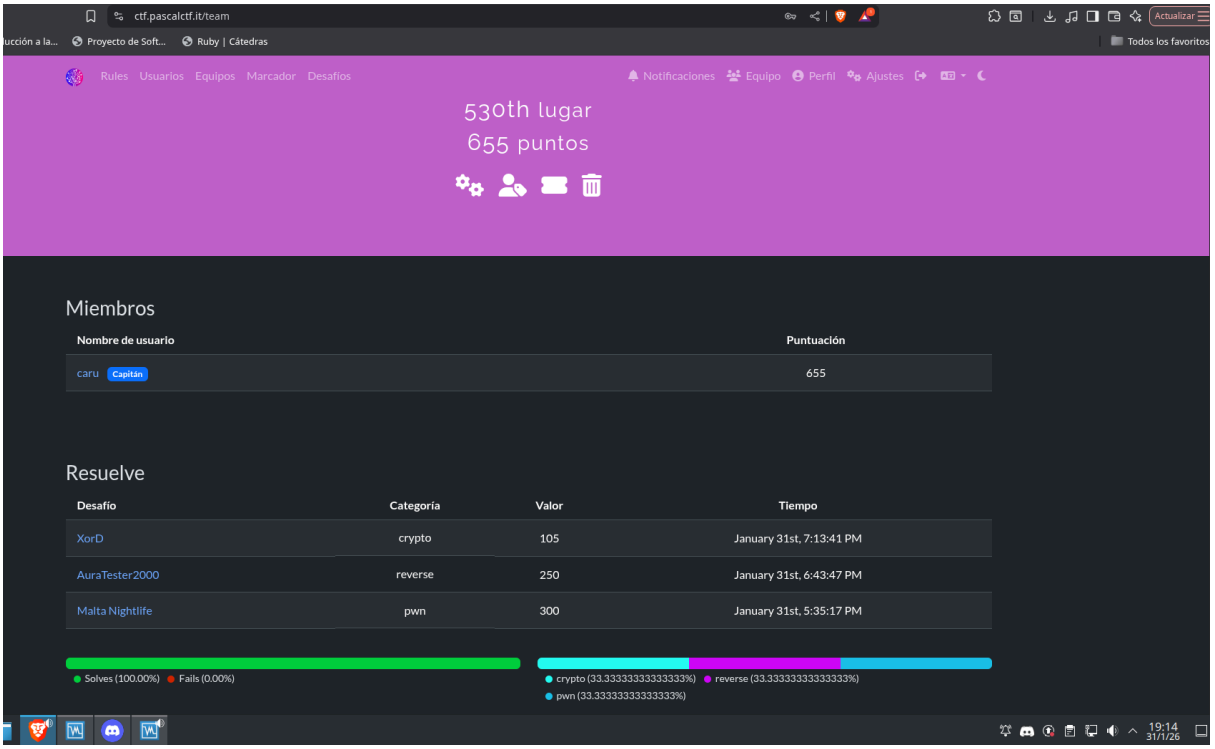
PascalCTF 2026

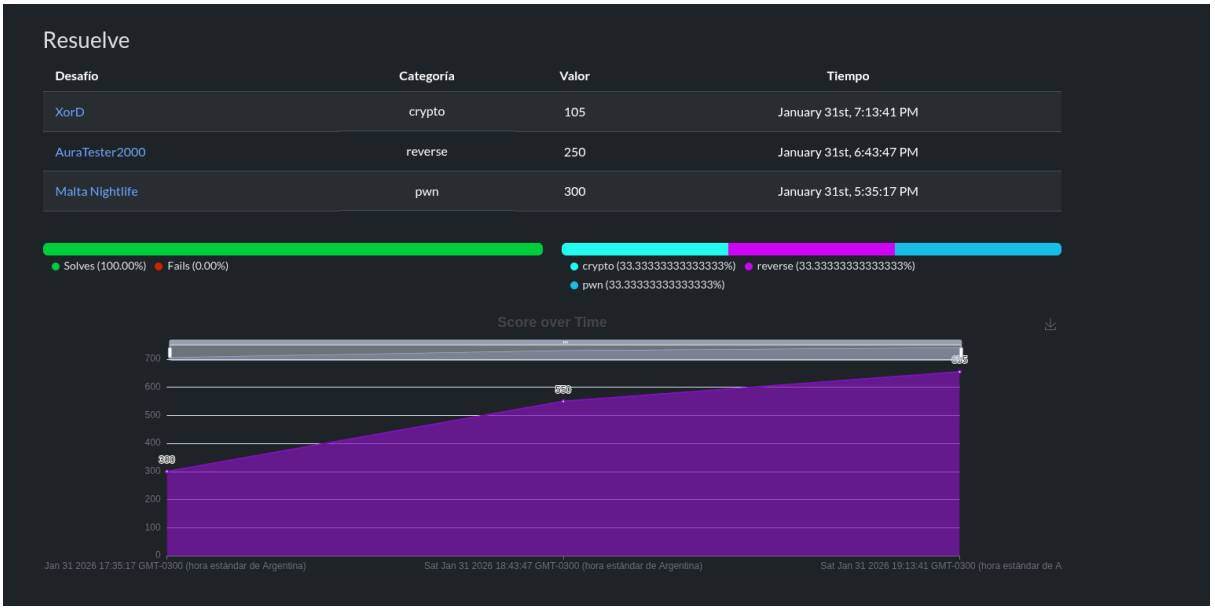
Registro PascalCTF 2026



Link a mi team https://ctf.pascalctf.it/teams/24

User: https://ctf.pascalctf.it/users?field=name&q=caru





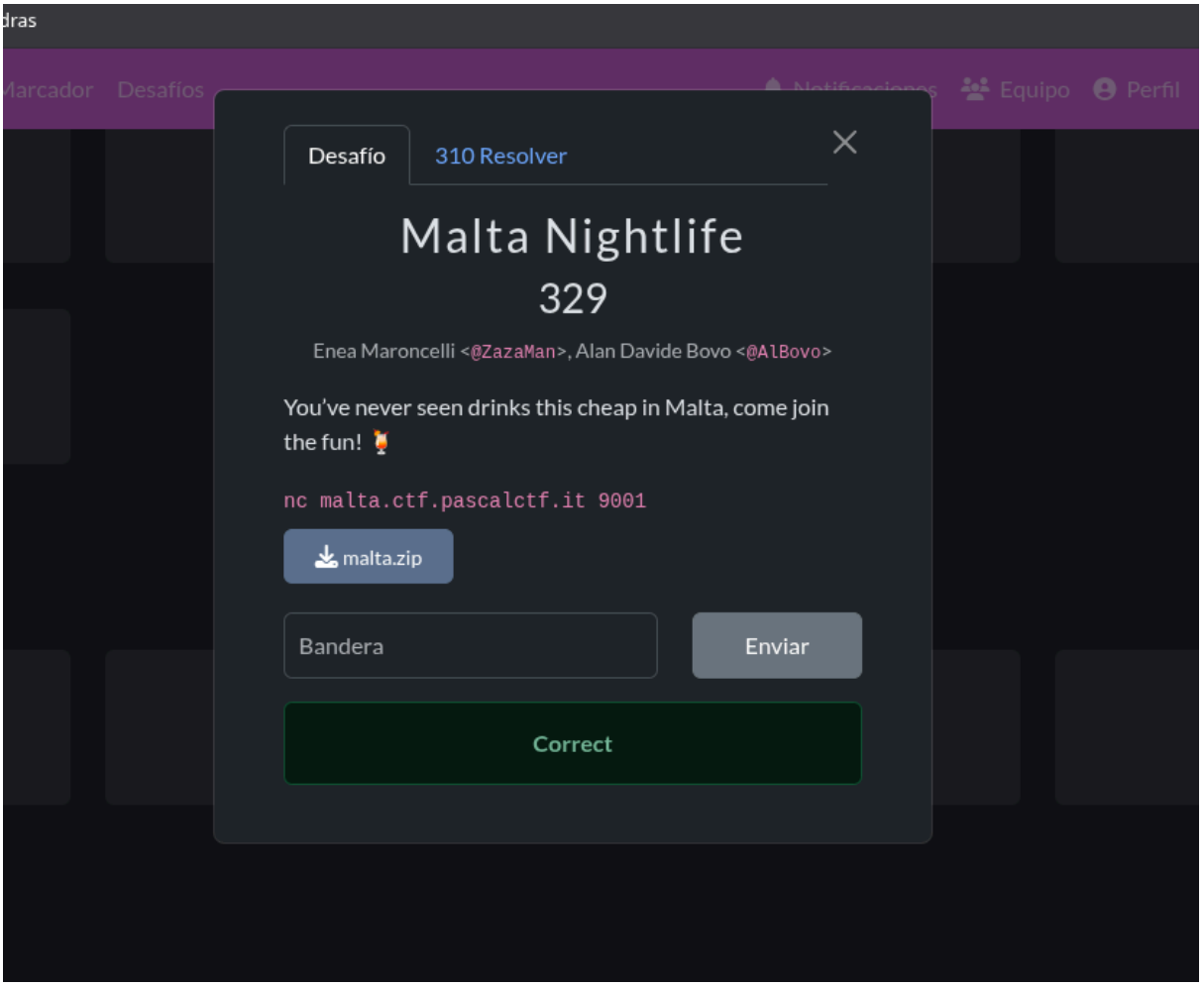
Olvide de sacar captura de mi score cada vez que realice un ejercicio, espero que estos graficos que el ctf provee sirvan para la consigna.

Malta Nightlife

Categoria: pwn

Descripcion: You've never seen drinks this cheap in Malta, come join the fun! 🍹

Archivos: Este ejercicio cuenta con un zip llamado malta, el cual al descomprimir nos da un binario.



Pasos:
1. Primero hay que crear la variable de entorno llamada FLAG para que el binario corra de manera local, esto lo pregunte al staff directamente, ya que no me parecía algo del ejercicio en sí sino como algo que no habían aclarado los creadores del enunciado, auqnue usando el comando strings se puede deducir.

```
export FLAG=falsa
```

2. Al ejecutar el programa vemos que nos ofrecen bebidas, cada una con un precio. También tenemos un saldo del cual no limita la cantidad de bebidas que podemos comprar. Además, hay una bebida que es la flag, está con un valor imposible desde el primer momento.

   Por lo que primero ingreso una cantidad que sé que no genera problemas.

```
MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMWOdkXWMMMMMMMMMMMMMMMMMMMMMMMMMMWXkdOWMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMN0KWWMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMWK0NMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
Welcome in Malta, here you're to buy some of the cheapest cocktails in the world!
Your balance is: 100 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: 1
How many drinks do you want? 4
You bought 4 Margarita for 24 € and the barman told you its secret recipe: Tequila & lime
Your balance is: 76 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: 2
How many drinks do you want? 13
You don't have enough money!
Your balance is: 76 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: 10
How many drinks do you want? 0
```

3. Luego para intentar romper este enunciado en lugar de intentar hacer un buffer overflow que es lo primero que intento siempre ingrese un número negativo en la selección de bebida, esto no funciono. Para esta captura volví a ejecutar el programa.

```
Welcome in Malta, here you're to buy some of the cheapest cocktails in the world!
Your balance is: 100 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: -6
Invalid choice
Your balance is: 100 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit
```

4. Luego hice algo parecido, pero en la cantidad de las bebidas, esto funcionó. Multiplico el valor de la bebida (6) por -1, esto generó que el programa haga, Balance = Balance - valor. Al hacer doble resta terminó incrementando el balance.

```
Select a drink: 5
How many drinks do you want? -1
You bought -1 Cosmopolitan for -6 € and the barman told you its secret recipe: Cranberry & vodka
Your balance is: 106 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: 9
How many drinks do you want? -30
You bought -30 Martini for -90 € and the barman told you its secret recipe: Classic & dry
Your balance is: 196 €
```

5. Aproveche este error e ingrese número más grandes para así obtener un saldo en el que pueda comprar la flag.

```
Welcome in Malta, here you're to buy some of the cheapest cocktails in the world!
Your balance is: 100 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: 2
How many drinks do you want?  -1000000000
You bought -1000000000 Mojito for -1705032704 € and the barman told you its secret recipe: Minty & refreshing
Your balance is: 1705032804 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit

Select a drink: 10
How many drinks do you want? 1
You bought 1 Flag for 1000000000 € and the barman told you its secret recipe: pascalCTF{St0p_dR1nKing_3ven_1f_it5_ch34p}
Your balance is: 705032804 €
1. Drink: Margarita for 6 €
2. Drink: Mojito for 6 €
3. Drink: Gin lemon for 5 €
4. Drink: PascalCTF26 for 6 €
5. Drink: Cosmopolitan for 6 €
6. Drink: Lavander Collins for 4 €
7. Drink: Japanese slipper for 5 €
8. Drink: Blue angel for 6 €
9. Drink: Martini for 3 €
10. Drink: Flag for 1000000000 €
11. Exit
```

Flag: pascalCTF{St0p_dR1nKing_3ven_1f_it5_ch34p}

AuraTester2000

Categoria: reverse

Descripcion: Will you be able to gain enogh aura?

Archivos: AuraTester2000.gyat, este es el código fuente del ejercicio que vamos a utilizar, es un text file que en realidad esta ofuscado, representa un programa de python.

Pasos

1. A primera vista vemos un menú de selección, en las opciones hay algunas que nos dan a entender que podemos ingresar datos.

```
Welcome to the AuraTester2000!
Here, we will make sure you have enough aura to join our alpha gang.
First of all, we need to know your name.
> Mateo
Welcome Mateo to the AuraTester2000!

1. Answer questions to gain or lose aura.

2. Check your current aura.

3. Take the final AuraTest to prove your worth.

4. Exit the AuraTester2000.
What do you want to do little Beta?
>
```

2. Intento ir directo a la prueba final donde asumo que está la flag, pero me dice que requiero puntos que se ganan en la opción 1.

```
What do you want to do little Beta?
> 3
You need more aura to even try the final AuraTest.


1. Answer questions to gain or lose aura.

2. Check your current aura.

3. Take the final AuraTest to prove your worth.

4. Exit the AuraTester2000.
What do you want to do little Beta?
>
```

3. Intento responder a ver que obtengo y perdí puntos. Así que paso al código a mirar como está hecho.

```
What do you want to do little Beta?
> 1
You choose to answer questions. Let's see how much aura you can gain!
Mateo, Do you believe in the power of aura? (yes/no)
> yes
Mateo, Do you a JerkMate account? (yes/no)
> yes
Mateo, Are you willing to embrace your inner alpha? (yes/no)
> yes
Mateo, Do you really like SHYNE from Travis Scott? (yes/no)
> yes
Sorry Mateo, you lost -500 aura points. Learn how to be a real Sigma!
```

4. Vemos el código fuente, vemos varias cosas hay palabras reservadas que claramente no son de ningún lenguaje, pero es similar a python, más que nada como parece definir funciones con bop, el uso del join y la manera de iterar con “mewing i in”.

```
glaze random ahh sonopazzo
glaze os ahh palle

words = ["tungtung","trallalero","filippo boschi","zaza","lakaka","gubbio","cucinato"]

phrase = " ".join(sonopazzo.sample(words,k=sonopazzo.randint(3, 5)))

steps = sonopazzo.randint(2, 5)
flag = palle.getenv("FLAG", "pascalCTF{REDACTED}")
bop encoder(phrase, steps):
    encoded_phrase = ""
    mewing i in huzz(0,len(phrase)):

        chat is this real phrase[i] twin " ":
            encoded_phrase rizz= phrase[i]

        yo chat i% steps twin 0:
            encoded_phrase rizz= str(ord(phrase[i]))

        only in ohio:
            encoded_phrase rizz= phrase[i]
```

5. Vemos la secciones clave del código:

   a. Esta función es la que nos muestra la preguntas cuando seleccionamos la opción de ganar o perder puntos. Vemos que son preguntas definidas, que tenemos una lista de sets y que nos pregunta todas las preguntas mediante una iteración.

      Dentro del bucle for representado por mewing vemos que tenemos lo que parece ser un if-elif de python. Si seleccionamos yes toma el primer valor del set que corresponda según la iteración. Por lo que analizando vemos que para ganar la mayor cantidad de puntos hay que responder “yes, no, yes, no”.

```
bop questions(name):
    gained_aura = 0
    questions = [
        "Do you believe in the power of aura? (yes/no)",
        "Do you a JerkMate account? (yes/no)",
        "Are you willing to embrace your inner alpha? (yes/no)",
        "Do you really like SHYNE from Travis Scott? (yes/no)",
    ]
    aura_values = [(150,-50), (-1000,50),(450,-80),(-100,50)]
    mewing i in huzz(len(questions)):
        print(f"{name}, {questions[i]}")
        answer = input("> ").strip().lower()
        chat is this real answer twin "yes":
            gained_aura rizz= aura_values[i][0]
        yo chat answer twin "no":
            gained_aura rizz=  aura_values[i][1]
    its giving gained_aura
```

Esta función es la que nos muestra la preguntas cuando seleccionamos la opción de ganar o perder puntos. Vemos que son preguntas definidas, que tenemos una lista de sets y que nos pregunta todas las preguntas mediante una iteración.
Dentro del bucle for representado por mewing vemos que tenemos lo que parece ser un if-elif de python. Si seleccionamos yes toma el primer valor del set que corresponda según la iteración. Por lo que analizando vemos que para ganar la mayor cantidad de puntos hay que responder “yes, no, yes, no”.

Esta función es la que nos muestra la preguntas cuando seleccionamos la opción de ganar o perder puntos. Vemos que son preguntas definidas, que tenemos una lista de sets y que nos pregunta todas las preguntas mediante una iteración.
Dentro del bucle for representado por mewing vemos que tenemos lo que parece ser un if-elif de python. Si seleccionamos yes toma el primer valor del set que corresponda según la iteración. Por lo que analizando vemos que para ganar la mayor cantidad de puntos hay que responder “yes, no, yes, no”.

a. En la función asociada a la prueba final (la opción 3) que se llama a la función encoder con 2 argumentos phrase y steps. Luego nos pide decodificar una frase que se genera con la función encoder, si obtenemos la frase original ganamos la flag.

```
bop aura_test(name):
    yap(f"{name}, you have reached the final AuraTest!")
    yap("If you want to win your prize you need to decode this secret phrase:",encoder(phrase, steps))

    guess = input("Type the decoded phrase to prove your worth:\n> ")
    chat is this real guess twin phrase:
        yap(f"Congratulations {name}! You have proven your worth and gained the ultimate aura!\nHere's your price:\n{flag}")
        exit()
    only in ohio:
        yap(f"Dont waste my time {name}, you failed the AuraTest. Try again but this time use all your aura!")

yap("Welcome to the AuraTester2000!\nHere, we will make sure you have enough aura to join our alpha gang.")
```

b. Este código parece representar el menú del inicio, porque asumo que es un bucle while el cual espera la selección de una opción, lo único a destacar es que para ingresar la opción de la prueba debemos tener 500 puntos, por lo que responder las preguntas es necesario.

```
let him cook(Aura):
    yap("\n\n1. Answer questions to gain or lose aura.\n\n2. Check your current aura.\n\n3. Take the fi
    choice = input("What do you want to do little Beta?\n> ")
    chat is this real (choice twin "1"):
        yap("You choose to answer questions. Let's see how much aura you can gain!")
        gained_aura = questions(name)
        chat is this real(aura sigma 0):
            yap(f"Congratulations {name}! You gained {gained_aura} aura points.")
        only in ohio:
            yap(f"Sorry {name}, you lost {gained_aura} aura points. Learn how to be a real Sigma!")
        aura += gained_aura

    yo chat(choice twin "2"):
        yap(f"Your current aura is {aura}.")
    yo chat(choice twin "3"):
        chat is this real(aura beta 500):
            yap("You need more aura to even try the final AuraTest.")
        only in ohio:
            aura_test(name)
    yo chat(choice twin "4"):
        yap("Exiting the AuraTester2000. Goodbye!")
        exit()
    only in ohio:
        yap("Invalid option. Please try again.")
```

c. La función encoder toma 2 parámetros phrase la cual se arma de manera aleatoria entre las palabras definidas en words, esto usando los que entiendo que es la importación de la librería random, renombrándola sonopazzo. El parámetro steps en un número random entre 2 y 5. En esta función se hace un for entre 0 y la longitud de la frase, luego pregunta si es un espacio vacío, si es así lo mantiene, ya que lo concatena con rizz= lo que significa +=, luego pregunta si el índice i divide la variable steps tiene resto 0, si es así toma una letra y la convierte en ASCII y luego a string para pegarla, si no se deja el elemento de la frase como está.

```
words = ["tungtung","trallalero","filippo boschi","zaza","lakaka","gubbio","cucinato"]

phrase = " ".join(sonopazzo.sample(words,k=sonopazzo.randint(3, 5)))

steps = sonopazzo.randint(2, 5)
flag = palle.getenv("FLAG", "pascalCTF{REDACTED}")
bop encoder(phrase, steps):
    encoded_phrase = ""
    mewing i in huzz(0,len(phrase)):

        chat is this real phrase[i] twin " ":
            encoded_phrase rizz= phrase[i]

        yo chat i% steps twin 0:
            encoded_phrase rizz= str(ord(phrase[i]))

        only in ohio:
            encoded_phrase rizz= phrase[i]


    its giving encoded_phrase
```

6. Intento ganar puntos siguiendo la lógica de yes, no, yes, no

```
What do you want to do little Beta?
> 1
You choose to answer questions. Let's see how much aura you can gain!
1, Do you believe in the power of aura? (yes/no)
> yes
1, Do you a JerkMate account? (yes/no)
> no
1, Are you willing to embrace your inner alpha? (yes/no)
> yes
1, Do you really like SHYNE from Travis Scott? (yes/no)
> no
Sorry 1, you lost 700 aura points. Learn how to be a real Sigma!


1. Answer questions to gain or lose aura.

2. Check your current aura.

3. Take the final AuraTest to prove your worth.

4. Exit the AuraTester2000.
What do you want to do little Beta?
> 2
Your current aura is 700.
```

7. Voy a la prueba final para ver si puedo entrar y que pasa.

```
What do you want to do little Beta?
> 3
1, you have reached the final AuraTest!
If you want to win your prize you need to decode this secret phrase: 102ilip112o bo115chi 122aza 103ubbi111 cuc105nato laka107a
Type the decoded phrase to prove your worth:
> hola
Dont waste my time 1, you failed the AuraTest. Try again but this time use all your aura!
```

8. Creo un script resolve.py el cual copia las mismas palabras que el código fuente y copia la función encoder. Esto con la finalidad de hacer fuerza bruta, ya que no sé que número será steps voy a probar codificando todas las combinaciones posibles de palabras hasta que una coincida con la frase codificada que me da el programa remoto y así saber cuáles son las palabras originales.

   a. Primero definí las palabras y la función.

```
import itertools
words = ["tungtung", "trallalero", "filippo boschi", "zaza", "lakaka", "gubbio", "cucinato"]
def encoder(phrase, steps):
    encoded_phrase = ""
    for i in range(len(phrase)):
        if phrase[i] == " ":
            encoded_phrase += phrase[i]
        elif i % steps == 0:
            encoded_phrase += str(ord(phrase[i]))
        else:
            encoded_phrase += phrase[i]
    return encoded_phrase
```

   b. Interactuo con el programa, primero contestando correctamente las preguntas

```
io.recvuntil(b'First of all, we need to know your name.\n')
io.sendline(b'Mateo')
io.recvuntil(b'What do you want to do little Beta?\n')
io.sendline(b'1')
for i in range(4):
    io.recvuntil(b'(yes/no)\n')
    if i % 2 == 0:
        io.sendline(b'yes')
    else:
        io.sendline(b'no')
```

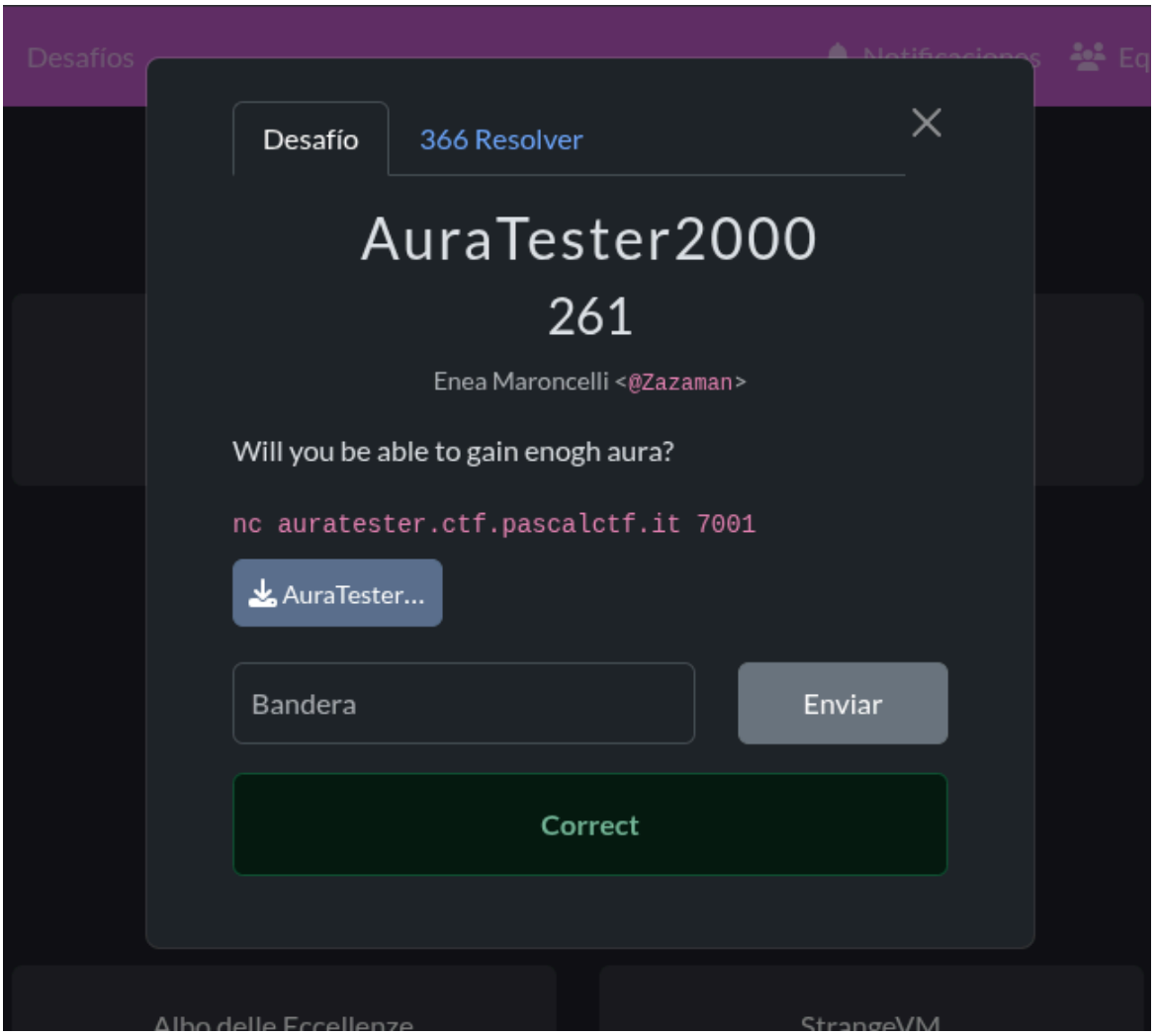
   c. Voy a la prueba final y recibo la frase codificada.

```
io.recvuntil(b'What do you want to do little Beta?\n')
io.sendline(b'3')
io.recvuntil(b'If you want to win your prize you need to decode this secret phrase: ')
secret = io.recvline().strip().decode()
decoded_phrase = ""
found = False
```

   d. Realizo la fuerza bruta, tomo el rango de 3 a 6 para que coincida con la cantidad posible de palabras que son entre 3 y 5. Luego dentro del for anidado armo las posibles frases. Por cada frase selecciono en el rango de 2 y 6 mi posible número de steps, como paso final codifico esa frase con ese step dentro de los bucles y pregunto si es igual a la frase codificada que me dio el programa, si es así corto el bucle y obtengo la frase decodificada para enviar al programa.

```python
for k in range(3, 6):
    if found: break
    for combo in itertools.permutations(words, k):
        phrase_candidate = " ".join(combo)
        for steps in range(2, 6):
            if encoder(phrase_candidate, steps) == secret:
                decoded_phrase = phrase_candidate
                found = True
                log.success(f"La frase es: {decoded_phrase}")
                break
io.recvuntil(b'Type the decoded phrase to prove your worth:\n')
io.sendline(decoded_phrase.encode())
io.interactive()
```

flag: pascalCTF{Y0u_4r3_th3_r34l_4ur4_f1n4l_b0s5}



XorD

Categoria: crypto

Archivos: Este desafio incluye 2 archivos, output.txt y xord.py, el primero es una salida que se genera a partir del segundo programa.

Pasos

1. Primero viendo el codigo de xord.py pude ver varias cosas.
    a. Utiliza la libreria random de python
    b. Se le setea una semilla fija al objeto random
    c. Se realiza un xor entre un caracter de la flag pasado a ASCII y una ket random

```python
import os
import random

def xor(a, b):
    return bytes([a ^ b])

flag = os.getenv('FLAG', 'pascalCTF{REDACTED}')
encripted_flag = b''
random.seed(1337)

for i in range(len(flag)):
    random_key = random.randint(0, 255)
    encripted_flag += xor(ord(flag[i]), random_key)

with open('output.txt', 'w') as f:
    f.write(encripted_flag.hex())
```

2. Buscando por donde encarar esto investigue la libreria random y que pasa al usar una semilla fija
3. Esta librería random como todo en informática simula ser aleatorio, aunque en realidad es pseudoaleatorio, todo lo que simule ser pseudoaleatorio necesita una base para generar un resultado, la vulnerabilidad en este caso es que la semilla es un valor fijo por lo que los valores que genere serán siempre los mismos.
4. Para resolver esto hice un script que hace lo siguiente:
    a. Importo la misma librería random, defino la salida que genera el programa original como una variable en bytes

```
import random

data = "cb35d9a7d9f18b3cfc4ce8b852edfaa2e83dcd4fb44a35909ff3395a
2656e1756f3b505bf53b949335ceec1b70e0"

encripted_flag = bytes.fromhex(data)
```

b. Defino la misma semilla para la libreria de random

```
random.seed(1337)
```

c. Por cada byte de la cadena encriptada genero la misma key que el programa

```
decrypted_flag = ""

for byte in encripted_flag:
    random_key = random.randint(0, 255)
```

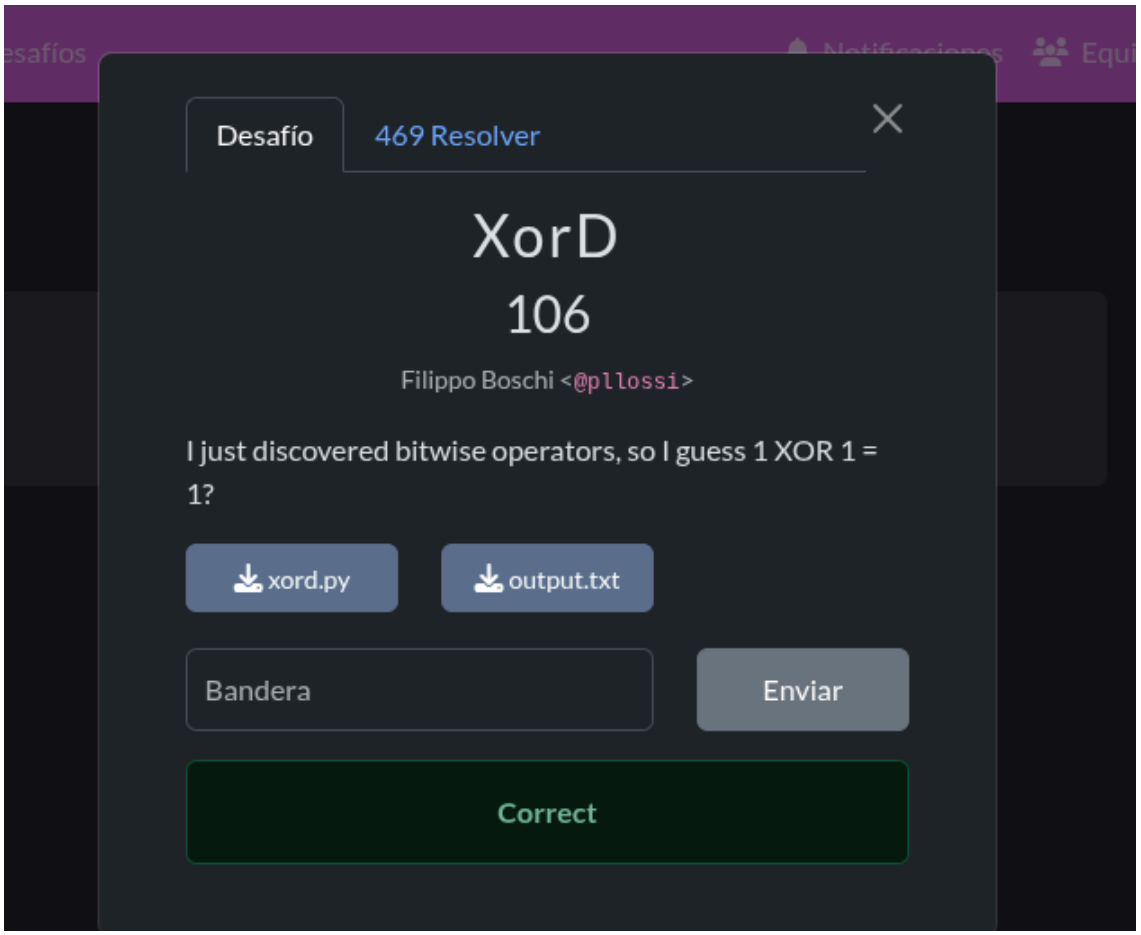
d. Por propiedad del XOR al hacer (A xor B) xor B obtengo siempre A

```
 decrypted_char = chr(byte ^ random_key)
```

e. Concateno y obtengo la flag

```
decrypted_flag += decrypted_char
```

flag: pascalCTF{1ts_4lw4ys_4b0ut_x0r1ng_4nd_s33d1ng}



Reflexión: Este fue el ctf más sencillo de los 2 que hice, pero no fueron aburridos, quizá el ejercicio que hice de pwn podría haber tenido una vuelta para hacerlo más interesante, pero en caso contrario al anterior el ejercicio de reverse de ofuscación me resulto simpático y poco mareante como él del anterior ctf, y en cuanto el ejercicio de crypto me resulto interesante que la vulnerabilidad sea la semilla para generar la “aleatoriedad”, ya que nunca había visto ningún ejercicio así.